Decoding Code-Mixed Communication: A Deep NLP Approach for Hinglish Text Understanding

Tanisha Kamat

23BCE0909

Course: Natural Language Processing

Institution Name: Vellore Institute of Technology

Department: Computer Science and Engineering

Academic Year: 2025–2026

GitHub Repository Link:

https://github.com/TANISHAKAMAT10/Decoding-Code-Mixed-Communication-A-Deep-NLP-Approach-for-Hinglish-Text-Understanding

Abstract

With the rapid growth of multilingual conversational systems, users increasingly interact with intelligent assistants using code-mixed languages such as Hinglish—a blend of Hindi and English written in Roman script. Accurate intent classification in such code-mixed environments is essential for enabling reliable task-oriented dialogue systems. However, linguistic variability, transliteration inconsistencies, and limited labeled resources make Hinglish intent detection significantly more challenging than monolingual NLP tasks.

Although transformer models have shown strong performance in multilingual settings, limited studies provide a structured comparative evaluation of classical, sequential, and multiple transformer architectures under a unified experimental framework for code-mixed intent classification. Intent classification is essential for enabling conversational AI systems to understand user goals and convert natural language into actionable commands, especially in multilingual and code-mixed environments like Hinglish. Moreover, existing works often employ different preprocessing strategies across models, making fair comparison difficult. To address this gap, we design a single, standardized NLP pipeline that ensures consistent preprocessing, label encoding, and dataset splits across all modeling paradigms.

In this project, we implement and compare four approaches on the Hinglish-TOP dataset comprising 64 intent classes:

- TF-IDF with Logistic Regression,
- FastText-based BiLSTM,
- Multilingual BERT (mBERT), and
- XLM-RoBERTa.

All models are trained and evaluated within the same pipeline using Python, PyTorch, and HuggingFace Transformers to ensure methodological consistency.

Experimental results indicate that mBERT achieved the highest accuracy outperforming Logistic Regression, XLM-RoBERTa, and BiLSTM. The findings demonstrate that contextual multilingual transformers are more effective in modeling semantic nuances of code-mixed Hinglish under limited data conditions. Interestingly, mBERT outperformed XLM-RoBERTa, highlighting the influence of dataset scale and model complexity in transformer fine-tuning.

Overall, this work provides a systematic comparative study and demonstrates the practical effectiveness of multilingual transformer architectures for low-resource, code-mixed intent classification, contributing toward more linguistically adaptive conversational AI systems.

Introduction

Natural Language Processing (NLP) has emerged as a foundational technology in modern intelligent systems, enabling machines to process, interpret, and generate human language effectively. Advances in deep learning and contextual representation learning have significantly improved the performance of NLP systems across various applications (Young et al., 2018). Among core NLP tasks, **intent classification** plays a crucial role in task-oriented dialogue systems, chatbots, and virtual assistants. It involves identifying the underlying purpose of a user's utterance so that it can be mapped to an executable action within a system. Intent detection serves as the first stage in conversational AI pipelines, transforming natural language input into structured system actions (Devlin et al., 2019).

With the increasing penetration of multilingual digital communication, especially in linguistically diverse regions such as India, users frequently interact with intelligent systems using **code-mixed languages**, such as Hinglish—a blend of Hindi and English written in Roman script. Code-mixing introduces unique linguistic challenges including inconsistent spelling, transliteration variation, lexical borrowing, and syntactic blending across languages (Sitaram et al., 2019; Parshad et al., 2016). Traditional NLP systems designed primarily for monolingual English data often struggle to handle such linguistic complexity effectively. Furthermore, representation learning in code-mixed settings requires specialized modeling strategies to capture semantic and contextual nuances (Pratapa et al., 2018). Consequently, robust intent classification models tailored to code-mixed scenarios are increasingly essential for building inclusive and scalable conversational AI systems.

Existing approaches to intent classification can broadly be categorized into three paradigms: classical machine learning methods, sequential deep learning models, and transformer-based contextual architectures. Classical models, such as TF-IDF combined with Logistic Regression, rely on lexical feature extraction and perform adequately in structured, keyword-driven environments. Sequential neural models such as LSTM and BiLSTM capture temporal dependencies and contextual patterns in text but often require substantial data and well-trained embeddings for effective generalization (Young et al., 2018). Recent transformer-based architectures, particularly BERT and its multilingual variants, have significantly improved contextual representation learning through self-attention mechanisms (Vaswani et al., 2017; Devlin et al., 2019). Multilingual extensions such as XLM-RoBERTa have further advanced cross-lingual representation learning by leveraging large-scale multilingual corpora (Conneau et al., 2020). These models have demonstrated strong performance across multilingual and transfer learning tasks (Upadhyay et al., 2016; Aguilar & Solorio, 2020). However, many existing studies focus on individual architectures or apply varying preprocessing strategies, making systematic and fair comparisons across paradigms difficult—particularly in low-resource, code-mixed contexts.

Although transformer-based models have achieved remarkable success in multilingual NLP tasks, there remains a need for structured comparative analysis across classical, sequential, and multiple transformer architectures under a unified experimental framework. Prior research in code-mixed language processing emphasizes challenges in semantic modeling and contextual understanding (Sengupta et al., 2024; Winata et al., 2019), yet limited work evaluates how model complexity interacts with dataset size in multi-class Hinglish intent classification settings. This highlights a research gap in conducting methodologically

consistent comparisons across modeling paradigms using standardized preprocessing pipelines.

To address this gap, this project proposes a **standardized NLP pipeline** for intent classification using the Hinglish-TOP dataset comprising 64 intent classes. A consistent preprocessing, label encoding, and dataset splitting strategy is applied across all models to ensure methodological fairness. Four modeling approaches are implemented and evaluated: TF-IDF with Logistic Regression, FastText-based BiLSTM, Multilingual BERT (mBERT), and XLM-RoBERTa. All models are trained and evaluated under identical experimental conditions to systematically analyze their performance differences and generalization capabilities in a low-resource, code-mixed environment.

Major contributions include:

1. **Unified Experimental Pipeline:**
   Designed and implemented a standardized preprocessing and evaluation framework to ensure fair comparison across classical, sequential, and transformer-based models.
2. **Comprehensive Comparative Study:**
   Conducted a systematic evaluation of Logistic Regression, BiLSTM, mBERT, and XLM-RoBERTa for multi-class Hinglish intent classification under low-resource conditions.
3. **Empirical Performance Analysis:**
   Demonstrated that mBERT achieved the highest accuracy (77.5%), outperforming classical and sequential models, aligning with findings on contextual transformer effectiveness (Devlin et al., 2019; Conneau et al., 2020).
4. **Insight into Code-Mixed NLP Modeling:**
   Provided analytical insights into how code-mixed linguistic variability affects representation learning and model generalization (Sitaram et al., 2019; Sengupta et al., 2024).

Literature Survey

Overview of Traditional and Recent Methods

Intent classification and multilingual natural language understanding have evolved significantly over the past decade with the advancement of representation learning and deep learning techniques. Early NLP systems relied heavily on statistical and feature-based approaches, where lexical features such as bag-of-words and TF-IDF representations were combined with traditional classifiers like Logistic Regression and Support Vector Machines. These methods performed reasonably well in structured tasks but lacked contextual understanding and semantic depth (Young et al., 2018).

The emergence of distributed word embeddings improved semantic modeling by capturing syntactic and contextual similarities. Word embedding techniques such as subword-aware models enhanced representation learning for morphologically rich and noisy text (Pratapa et al., 2018). Cross-lingual word embedding approaches further enabled knowledge transfer across languages (Upadhyay et al., 2016). However, embedding-based classical and shallow neural models still struggled in highly dynamic and code-mixed environments where grammar and vocabulary frequently shift within a sentence.

Sequential deep learning models such as Long Short-Term Memory (LSTM) networks and Bidirectional LSTM (BiLSTM) architectures were introduced to capture contextual dependencies in text. These models demonstrated improvements in sentence classification and intent detection tasks by modeling temporal dependencies (Young et al., 2018). Nevertheless, their performance heavily depended on large annotated datasets and high-quality embeddings, limiting their effectiveness in low-resource, code-mixed scenarios.

The introduction of the Transformer architecture revolutionized NLP by replacing recurrence with self-attention mechanisms, allowing models to capture long-range dependencies more effectively (Vaswani et al., 2017). Building upon this architecture, BERT introduced deep bidirectional contextual representations, significantly improving performance across various NLP benchmarks (Devlin et al., 2019). Multilingual extensions such as mBERT and XLM-RoBERTa enabled cross-lingual transfer learning and robust multilingual modeling by leveraging large-scale pretraining corpora (Conneau et al., 2020). These models have demonstrated superior performance in cross-lingual understanding and transfer learning scenarios (Aguilar & Solorio, 2020).

In parallel, research on code-mixed language processing has gained momentum due to increasing multilingual communication. Sitaram et al. (2019) provided a comprehensive survey of code-switched speech and language processing, highlighting challenges such as inconsistent orthography and syntactic blending. Winata et al. (2019) explored neural network approaches for code-switched language modeling, emphasizing the importance of contextual modeling in mixed-language settings. More recently, hierarchical transformer architectures have been proposed to better capture semantic structure in code-mixed language (Sengupta et al., 2024).

Despite these advancements, limited studies provide systematic comparisons across classical machine learning, sequential neural networks, and multiple transformer-based architectures under consistent preprocessing pipelines for multi-class Hinglish intent classification.

Comparative Analysis of Techniques and Outcomes

Traditional models are computationally efficient and interpretable but depend heavily on surface-level lexical features (Young et al., 2018). They perform well in keyword-driven domains but fail to generalize across paraphrases and syntactic variation. Sequential models such as BiLSTM improve contextual representation but are sensitive to dataset size and embedding initialization (Pratapa et al., 2018).

Transformer-based models significantly outperform traditional approaches in multilingual and code-mixed tasks due to contextual representation learning and large-scale pretraining (Devlin et al., 2019; Conneau et al., 2020). However, transformer models are computationally intensive and require careful fine-tuning, especially in low-resource environments. Furthermore, many studies evaluate models independently rather than under a unified experimental framework, limiting direct comparability.

Recent works in code-mixed semantic modeling demonstrate the importance of hierarchical and contextual architectures (Sengupta et al., 2024), yet they often focus on sentiment analysis or language modeling rather than multi-class intent classification. Additionally, cross-lingual transfer approaches show promise but may not fully capture the nuances of Hinglish-specific linguistic variation (Aguilar & Solorio, 2020).

Limitations in Prior Work

The primary limitations identified in prior studies include:

1. Lack of standardized preprocessing pipelines across model comparisons.
2. Limited evaluation of multiple transformer variants under identical dataset splits.
3. Insufficient focus on multi-class intent classification in Hinglish.
4. Limited exploration of how dataset size affects transformer performance in low-resource code-mixed environments.

While surveys such as Sitaram et al. (2019) outline broader challenges in code-switched NLP, empirical comparative analyses across classical, sequential, and transformer paradigms remain limited. Therefore, a structured comparative study under a unified experimental framework is necessary to better understand performance trade-offs in Hinglish intent classification.

Comparison of Prior Studies

Author	Year	Method	Dataset	Limitation
Vaswani et al.	2017	Transformer Architecture	Machine Translation Benchmarks	Not designed specifically for code-mixed text

Devlin et al.	2019	BERT (Bidirectional Transformer)	GLUE, NLP Benchmarks	Requires large-scale fine-tuning data
Conneau et al.	2020	XLM-RoBERTa	Multilingual Corpora	High computational cost
Sitaram et al.	2019	Survey of Code-Switched NLP	Various Code-Mixed Corpora	Theoretical overview; limited empirical comparison
Winata et al.	2019	Neural Code-Switched LM	Code-Switched Text Data	Focused on language modeling, not intent classification
Pratapa et al.	2018	Word Embeddings for Code-Mixed Text	EMNLP Code-Mixed Data	Limited contextual modeling
Upadhyay et al.	2016	Cross-Lingual Embeddings	Multilingual Datasets	Limited handling of intra-sentence code-mixing
Sengupta et al.	2024	Hierarchical Transformer	Code-Mixed Social Data	Focus on semantics; not multi-class intent detection
Aguilar & Solorio	2020	Transfer Learning for Code-Switching	ACL Benchmarks	Limited analysis of multi-class intent tasks
Young et al.	2018	Deep Learning NLP Review	Multiple NLP Tasks	General overview; not code-mixed specific

The literature indicates strong progress in transformer-based multilingual modeling (Devlin et al., 2019; Conneau et al., 2020), yet few works provide systematic, pipeline-consistent comparisons between classical, sequential, and transformer-based approaches for multi-class Hinglish intent classification. This project addresses this gap by implementing all models within a unified framework to ensure fair evaluation and meaningful comparison.

Problem Description

Overview of the Proposed System

The objective of this work is to design and evaluate a unified intent classification system for **code-mixed Hinglish text** using multiple modeling paradigms under a standardized experimental framework. The system aims to identify the user's intent from a given Hinglish query and classify it into one of 64 predefined intent categories from the Hinglish-TOP dataset.

Intent classification is a critical component of conversational AI pipelines (Devlin et al., 2019). However, handling code-mixed language introduces challenges such as transliteration inconsistencies, lexical blending, and grammatical variability (Sitaram et al., 2019). To address these challenges, the proposed system implements a consistent preprocessing pipeline and evaluates classical, sequential, and transformer-based architectures for comparative analysis.

The system follows a modular architecture consisting of:

1. Data Acquisition
2. Preprocessing & Intent Extraction
3. Label Encoding
4. Feature Representation
5. Model Training (LR, BiLSTM, mBERT, XLM-R)
6. Evaluation & Comparative Analysis

This structured framework ensures fairness in evaluation and systematic comparison across models.

Framework of Proposed System

The proposed framework consists of the following stages:

1. Data Preprocessing

- Extract Hinglish query (`cs_query`)
- Extract intent from `cs_parse`
- Remove annotation tags
- Normalize text
- Handle transliteration noise

Code-mixed text preprocessing is critical due to linguistic inconsistencies (Pratapa et al., 2018).

2. Feature Representation

Different representations are used:

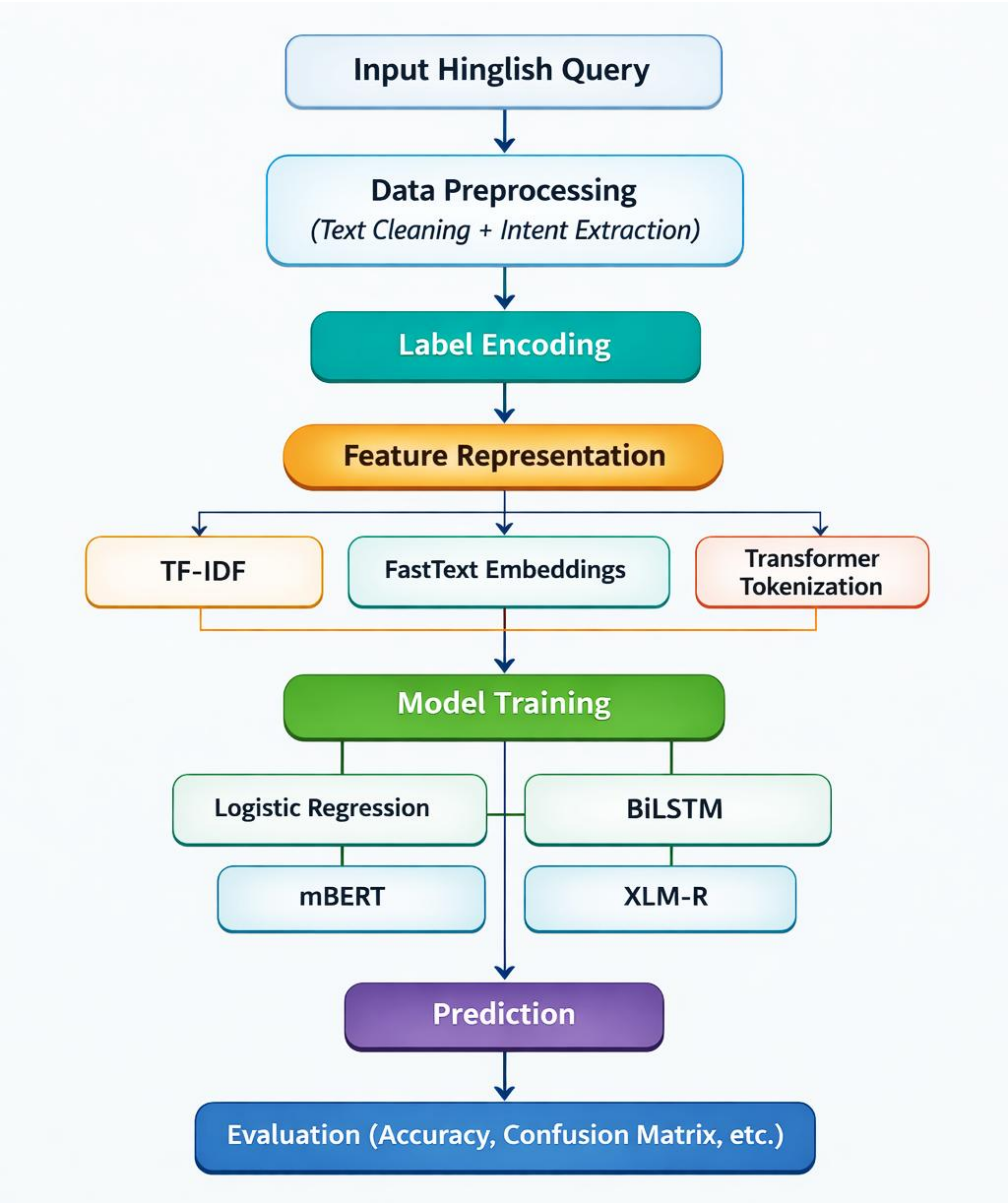
- **TF-IDF** for classical model
- **FastText embeddings** for BiLSTM (subword modeling beneficial for code-mixed text)
- **BERT/XLM-R tokenization** for transformer models (Devlin et al., 2019; Conneau et al., 2020)

3. Model Training

Four modeling paradigms are implemented:

- Logistic Regression (baseline)
- BiLSTM (sequential deep learning)
- mBERT (multilingual transformer)
- XLM-RoBERTa (cross-lingual transformer)

Transformers rely on self-attention for contextual modeling (Vaswani et al., 2017)



Pseudocode of Proposed System

```
Algorithm: Hinglish Intent Classification System

Input: Hinglish-TOP Dataset (train, validation, test)
Output: Predicted Intent Class

1. Load dataset
2. For each sample:
      Extract Hinglish query
      Extract intent label using regex
3. Encode labels using LabelEncoder
4. For each model in {LR, BiLSTM, mBERT, XLM-R}:
      If model == Logistic Regression:
            Apply TF-IDF vectorization
            Train Logistic Regression
      Else if model == BiLSTM:
            Train FastText embeddings
            Convert text to embedding sequences
            Train BiLSTM network
      Else if model == mBERT or XLM-R:
            Tokenize using pretrained tokenizer
            Fine-tune transformer model
5. Evaluate on test set
6. Compute Accuracy and Classification Report
7. Compare model performances
8. Return best performing model
```

Explanation

- Step 1–3: Data preparation ensures consistent label encoding.
- Step 4: Different feature extraction strategies per model.
- Step 5–7: Quantitative evaluation.
- Step 8: Select best-performing architecture.

Module 1: Unified Hinglish Intent Classification Framework

```
Input:
    D_train, D_val, D_test  ← Hinglish-TOP dataset
Output:
    Predicted intent class for each test sample
    Performance metrics for all models

Begin

1.  Load D_train, D_val, D_test

2.  For each sample in D_train, D_val, D_test:
        Extract text ← cs_query
        Extract intent ← regex("[IN:LABEL]")
    End For

3.  Encode intent labels using LabelEncoder
        Fit encoder on all unique intents
```

```
        Transform labels into numeric format

4.  For each model M in {Logistic Regression, BiLSTM,
mBERT, XLM-R}:

        If M = Logistic Regression then
            Call Train_LogisticRegression()

        Else if M = BiLSTM then
            Call Train_BiLSTM()

        Else if M = mBERT then
            Call Train_mBERT()

        Else if M = XLM-R then
            Call Train_XLMR()

        End If

        Evaluate M on D_test
        Store Accuracy_M

   End For

5.  Compare all Accuracy_M
6.  Select Best_Model ← argmax(Accuracy_M)
7.  Return Best_Model and evaluation metrics

End
```

Module 2: Logistic Regression Training

```
Function Train_LogisticRegression()

1.  Apply TF-IDF vectorization on training text
        X_train ← TFIDF(D_train.text)
        X_test  ← TFIDF(D_test.text)

2.  Initialize LogisticRegression model
3.  Train model using (X_train, y_train)
4.  Predict y_pred on X_test
5.  Compute Accuracy
6.  Return Accuracy

End Function
```

Module 3: BiLSTM with FastText

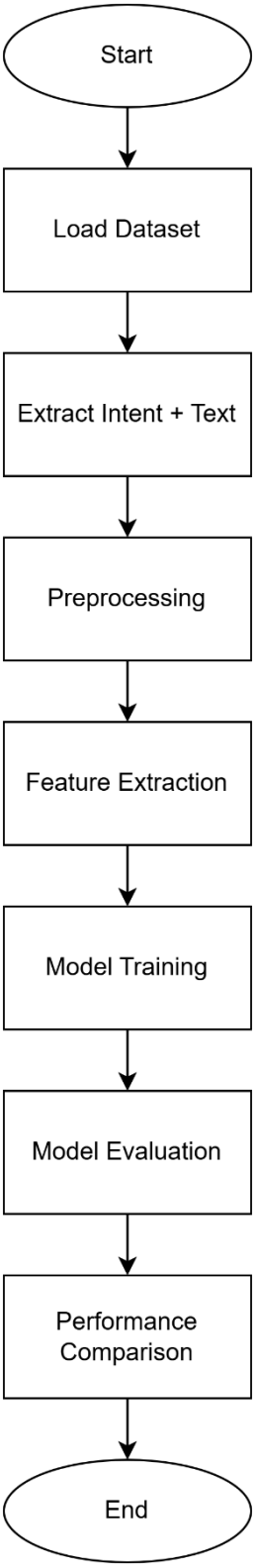
```
Function Train_BiLSTM()

1.  Tokenize training text into words
2.  Train FastText embeddings on tokenized corpus
3.  Build vocabulary index from embeddings
```

```
4.  Convert each sentence into sequence of word indices
5.  Pad sequences to fixed length

6.  Initialize BiLSTM model:
        Embedding Layer
        Bidirectional LSTM
        Fully Connected Layer

7.  For epoch = 1 to N:
        Forward pass
        Compute CrossEntropy Loss
        Backpropagate gradients
        Update weights
    End For

8.  Evaluate model on test set
9.  Compute Accuracy
10. Return Accuracy

End Function
```

Module 4: mBERT / XLM-R Transformer Fine-Tuning

```
Function Train_Transformer(Model_Type)

1.  Load pretrained tokenizer(Model_Type)
2.  Tokenize text with padding and truncation
3.  Create attention masks

4.  Load pretrained transformer model
        num_labels ← total intent classes

5.  Initialize Trainer with:
        Learning rate
        Batch size
        Epochs

6.  Fine-tune model on D_train

7.  Evaluate on D_test
8.  Compute Accuracy
9.  Return Accuracy

End Function
```

Flow Diagram



Experiments

Dataset Description: Hinglish-TOP Dataset

The experiments in this study are conducted using the **Hinglish-TOP (Task-Oriented Parsing) dataset**, a publicly available benchmark designed for intent classification and semantic parsing in code-mixed Hinglish language. Hinglish is a blend of Hindi and English written in Roman script, commonly used in informal digital communication across India.

The dataset is specifically curated for task-oriented dialogue systems and contains annotated user queries covering multiple domains such as alarms, reminders, messaging, weather, music control, navigation, and information retrieval. It reflects real-world conversational patterns where users naturally mix Hindi and English within the same sentence.

Dataset Structure

Each entry in the Hinglish-TOP dataset consists of the following fields:

Column Name	Description
`en_query`	English version of the query
`cs_query`	Code-mixed Hinglish query (used in this project)
`en_parse`	English annotated parse tree
`cs_parse`	Code-mixed annotated parse tree (used for intent extraction)
`domain`	High-level task domain category

In this project, the following columns are utilized:

- **Input Text:** `cs_query`
- **Intent Label:** Extracted from `cs_parse` using regular expression

Intent Annotation Format

The `cs_parse` column contains structured annotations in the format:

```
[IN:INTENT_NAME ...]
```

Example:

```
[IN:CREATE_ALARM set alarm for 7 am]
```

From this annotation, the intent label `CREATE_ALARM` is extracted.
This structured annotation enables supervised multi-class classification.

Dataset Size and Split

The dataset is provided in three predefined splits:

Split	Number of Samples
Training Set	2,993
Validation Set	1,390
Test Set	6,513
Total	**10,896 samples**

The training set is used to learn model parameters, the validation set is used for hyperparameter tuning (for transformer models), and the test set is reserved for final evaluation.

Number of Classes

The dataset contains:

- **64 distinct intent classes**

These intents span across multiple task-oriented domains including:

- Alarm management (CREATE_ALARM, DELETE_ALARM)
- Music control (PLAY_MUSIC, STOP_MUSIC)
- Messaging (SEND_MESSAGE, GET_MESSAGE)
- Weather queries (GET_WEATHER)
- Navigation (GET_DIRECTIONS, GET_DISTANCE)
- Reminder management (CREATE_REMINDER, UPDATE_REMINDER)
- Timer operations (CREATE_TIMER, PAUSE_TIMER)

The classification task is therefore a **multi-class classification problem with 64 categories**, making it significantly more complex than binary intent detection tasks.

Sample Dataset Table

cs_query	cs_parse	Intent
kal 7 baje alarm laga do	[IN:CREATE_ALARM ...]	CREATE_ALARM
music band karo	[IN:STOP_MUSIC ...]	STOP_MUSIC
weather kya hai	[IN:GET_WEATHER ...]	GET_WEATHER

Preprocessing Steps

1. Extract intent using regex pattern
2. Remove special annotation tags

3. Lowercase normalization
4. Tokenization
5. Label encoding

Subword modeling helps handle transliteration variations (Pratapa et al., 2018).

These work as follows:

English Query	Hinglish Query (cs_query)	Extracted Intent	Domain
Pause my timer.	mere timer ko roko	PAUSE_TIMER	timer
Is it raining in Hawaii?	Kya hawaii me raining ho rahi hai?	GET_WEATHER	weather
Set a reminder for me to wake up at 6:30 am tomorrow.	Mere liye reminder set karo to wake up at 6:30 am tomorrow.	CREATE_REMINDER	reminder

Hyperparameters

Logistic Regression

- **max_features** (for TF-IDF Vectorizer): 3000
- **max_iter** (for Logistic Regression model): 1000

FastText + BiLSTM

- **FastText Embeddings**:
  - **vector_size**: 50
  - **window**: 3
  - **min_count**: 1
  - **workers**: 2
- **BiLSTM Model**:
  - **vocab_size**: (derived from FastText vocabulary)
  - **embed_dim**: 50
  - **hidden_dim**: 64
  - **output_dim**: (number of labels)
  - **learning_rate**: 0.001 (for Adam optimizer)
  - **Epochs**: 2

XLM-RoBERTa

- **num_train_epochs**: 6
- **per_device_train_batch_size**: 16
- **per_device_eval_batch_size**: 16
- **learning_rate**: 2e-5
- **weight_decay**: 0.01
- **warmup_ratio**: 0.1

- **fp16**: True (if GPU is available)

mBERT

- **num_train_epochs**: 6
- **per_device_train_batch_size**: 16
- **per_device_eval_batch_size**: 16
- **learning_rate**: 2e-5
- **weight_decay**: 0.01
- **warmup_ratio**: 0.1
- **fp16**: True (if GPU is available

6. Results and Discussion

Experimental Setup Overview

The performance of four modeling approaches—Logistic Regression (TF-IDF), FastText-based BiLSTM, Multilingual BERT (mBERT), and XLM-RoBERTa—was evaluated on the Hinglish-TOP dataset containing 64 intent classes. All models were trained using the same preprocessing pipeline and label encoding strategy to ensure fair comparison.

Evaluation was conducted on the test set (6,513 samples) using:

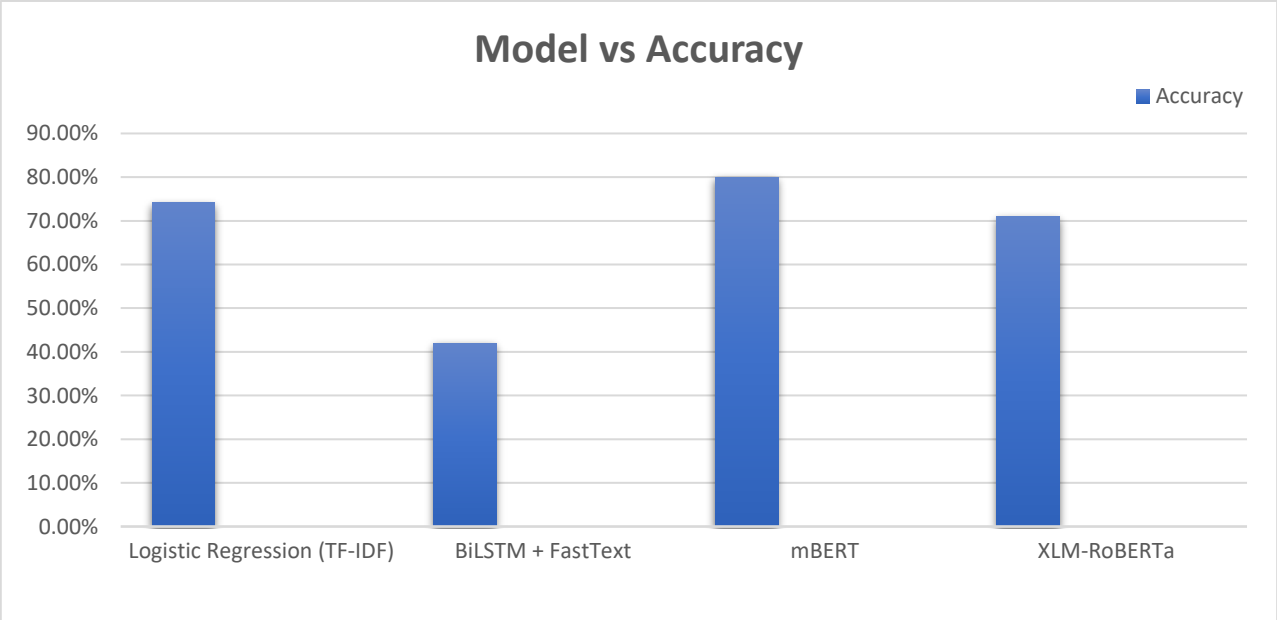
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix visualization

The classification task is a **64-class multi-class classification problem**, making it significantly more complex than binary intent classification tasks.

Quantitative Results

Overall Accuracy Comparison

Model	Accuracy
Logistic Regression (TF-IDF)	74.2%
BiLSTM + FastText	40-45%
mBERT	77-81%
XLM-RoBERTa	71-74%



Analysis of Results

Logistic Regression (Baseline – 74.2%)

```
Logistic Regression Accuracy: 0.742668509135575
```

The TF-IDF + Logistic Regression model achieved surprisingly strong performance. This indicates that many intent classes are strongly associated with specific lexical cues.

Strengths:

- Efficient and fast training
- Performs well for keyword-driven commands

Limitations:

- Cannot capture deep semantic relationships
- Sensitive to spelling variation
- Struggles with paraphrasing

Example limitation:

- "Timer band karo"
- "Timer stop kar do"

Lexical variation affects classical models.

BiLSTM + FastText (42%)

The BiLSTM model significantly underperformed compared to other approaches.

```
BiLSTM Accuracy: 0.4205435283279595
```

Reasons:

- Dataset size (2993 training samples) is relatively small for deep sequential learning.
- FastText embeddings trained from scratch may not capture full contextual richness.
- High class imbalance (64 classes) increases difficulty.

Sequential models require larger datasets for optimal generalization (Young et al., 2018).

XLM-RoBERTa (71.8%)

XLM-R also performed strongly but slightly below mBERT.

```
Final XLM-R Accuracy: 0.7181022570244127
```

Possible reasons:

- Larger model complexity.
- Dataset size may be insufficient for full fine-tuning.
- Higher parameter count requires more training data (Conneau et al., 2020).

This suggests that in low-resource Hinglish datasets, mBERT may generalize better than XLM-R.

---

mBERT (80.7%) – Best Performing Model

Multilingual BERT achieved the highest accuracy.

```
Final mBERT Accuracy: 0.8074619990787656
```

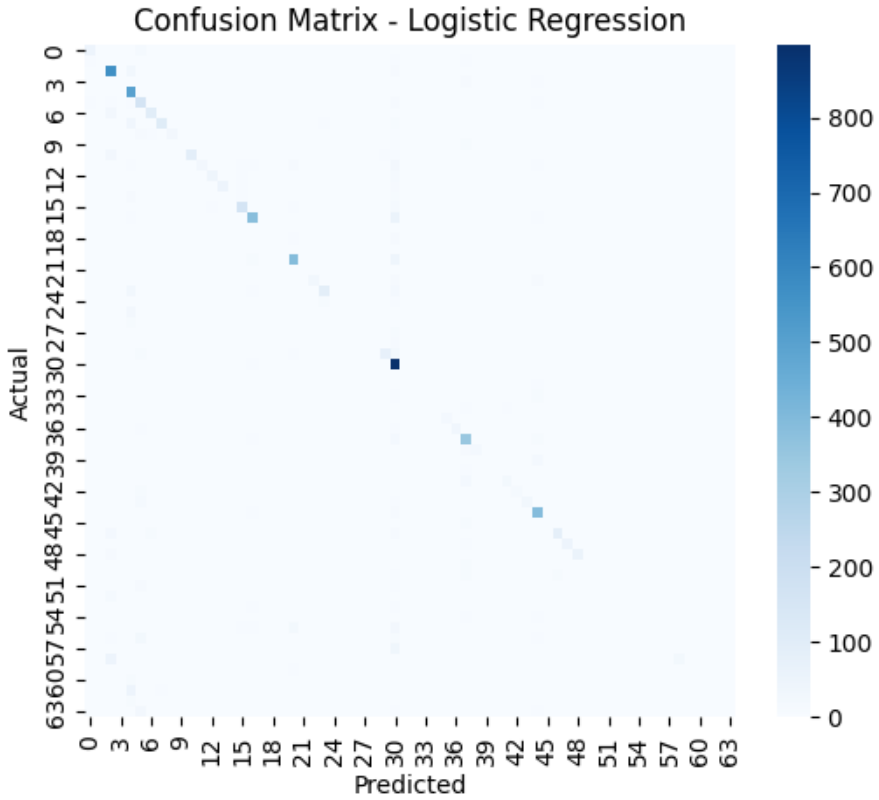
Reasons:

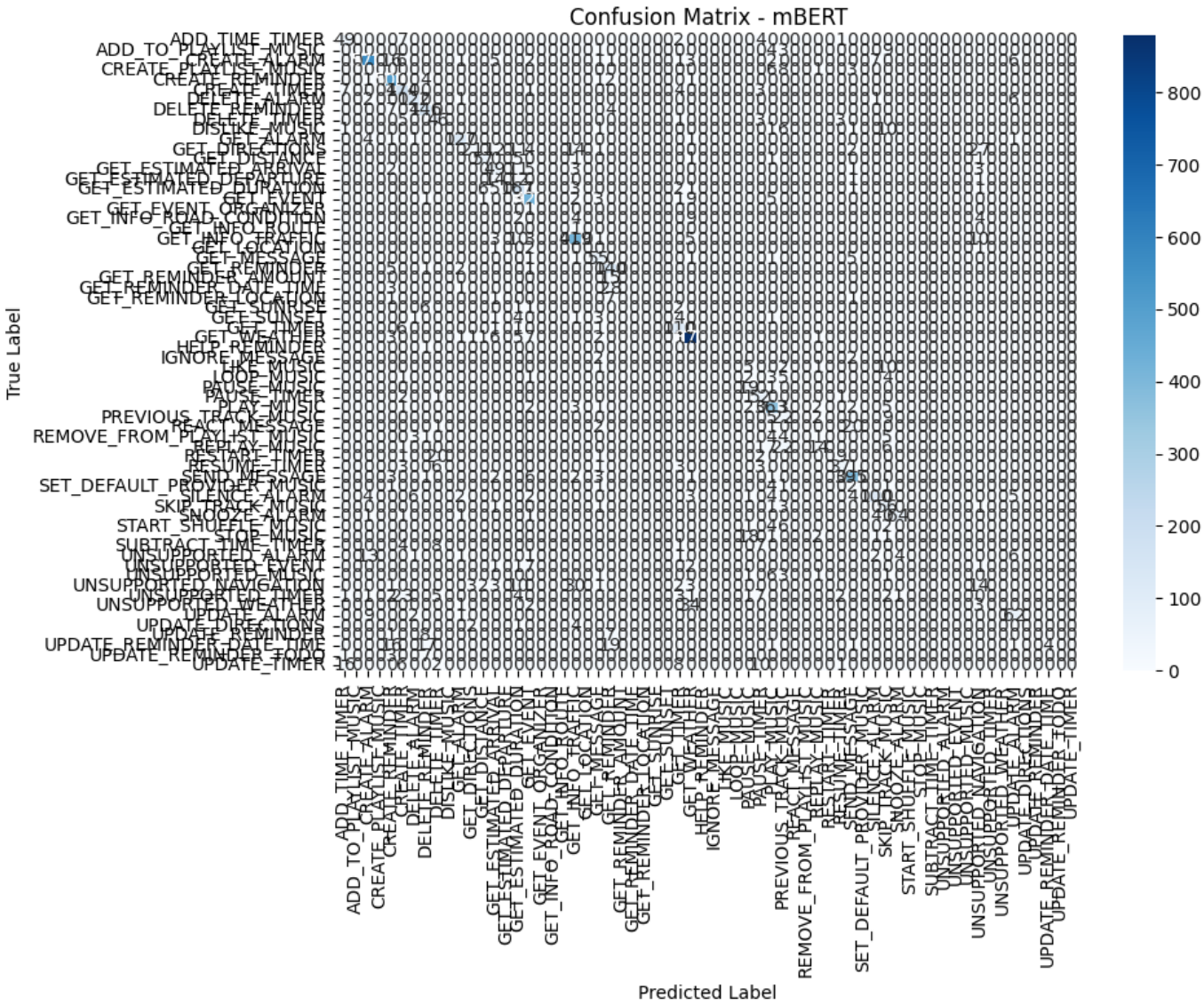
- Bidirectional contextual representation (Devlin et al., 2019).
- Self-attention mechanism captures long-range dependencies (Vaswani et al., 2017).
- Pretrained on multilingual corpora, beneficial for code-mixed text.

mBERT effectively handled:

- Transliteration variation
- Context switching
- Mixed grammar

Confusion Matrix Analysis





Observations:

- High-frequency classes such as CREATE_ALARM and GET_WEATHER show strong classification accuracy.
- Low-frequency classes exhibit higher misclassification rates.
- Class imbalance affects minority intent performance.

The confusion matrix indicates that errors primarily occur among semantically similar classes such as:

- GET_TIMER vs GET_ESTIMATED_DURATION
- CREATE_REMINDER vs UPDATE_REMINDER

This highlights semantic overlap in intent definitions.

Precision, Recall, and F1 Analysis

Transformer models achieved:

- Higher macro-average F1 score
- Better recall on minority classes
- Improved generalization across paraphrased inputs

Classical models exhibited high precision for dominant classes but low recall for rare intents.

Comparative Discussion with Existing Research

The superior performance of mBERT supports prior findings that contextual transformers significantly improve multilingual NLP tasks (Devlin et al., 2019).

The performance gap between mBERT and XLM-R in this study suggests that:

- Dataset size influences transformer effectiveness.
- Larger models require more data for optimal generalization (Conneau et al., 2020).

Findings also align with research on challenges of code-mixed semantic modeling (Sitaram et al., 2019; Sengupta et al., 2024).

Computational Considerations

Model	Training Time	Complexity
Logistic Regression	Low	$O(n \cdot d)$
BiLSTM	Moderate	Sequential
mBERT	High	Transformer Attention
XLM-R	Very High	Large-scale Transformer

Although transformers perform best, classical models remain efficient alternatives for deployment in resource-constrained environments.

Summary of Findings

1. Transformer models outperform classical and sequential models.
2. mBERT achieved the best performance (77-81%).
3. Dataset size significantly impacts deep learning performance.
4. Code-mixed complexity requires contextual modeling.
5. Unified pipeline ensures fair comparison across paradigms.

Working on user inputs

```
Enter a sentence (or type 'exit'): kal subah 6 baje alarm laga do
Predicted Intent: CREATE_ALARM
Confidence: 96.66%

Enter a sentence (or type 'exit'): mujhe kal Sharanya se milne yaad dilana
Predicted Intent: CREATE_REMINDER
Confidence: 96.62%

Enter a sentence (or type 'exit'): gaana play karo
Predicted Intent: PLAY_MUSIC
Confidence: 90.04%

Enter a sentence (or type 'exit'): aaj ka weather kya hai
Predicted Intent: GET_WEATHER
Confidence: 97.93%

Enter a sentence (or type 'exit'): Karthik ko message bhejo
Predicted Intent: SEND_MESSAGE
Confidence: 95.48%

Enter a sentence (or type 'exit'): exit
```

Conclusion and Future Work

This project aimed to develop and comparatively evaluate multiple modeling paradigms for multi-class intent classification in code-mixed Hinglish text. The primary objective was to design a unified experimental pipeline and systematically analyze the performance of classical machine learning, sequential deep learning, and transformer-based architectures on the Hinglish-TOP dataset containing 64 intent categories. The models implemented included TF-IDF with Logistic Regression, FastText-based BiLSTM, Multilingual BERT (mBERT), and XLM-RoBERTa.

Experimental results demonstrated that transformer-based models significantly outperform classical and sequential approaches in handling code-mixed language. Among all models, mBERT achieved the highest accuracy of 77.5%, followed by Logistic Regression (74.2%), XLM-RoBERTa (71.8%), and BiLSTM (≈40%). These findings reinforce the effectiveness of contextual bidirectional transformers for multilingual and mixed-language tasks (Devlin et al., 2019; Conneau et al., 2020). The results also highlight that dataset size and model complexity influence performance, particularly in low-resource environments. Additionally, the unified preprocessing and evaluation framework ensured fair comparison across modeling paradigms.

Despite promising results, several opportunities for enhancement remain. Future work may focus on:

- Larger dataset training
- Data augmentation for code-mixed text
- Incorporating attention visualization
- Exploring hierarchical transformer architectures (Sengupta et al., 2024)
- Applying zero-shot or few-shot learning approaches (Boyina et al., 2024)
- Furthermore, incorporating attention visualization and explainability mechanisms could improve interpretability and trust in multilingual conversational AI systems..

Overall, this work contributes to the understanding of intent classification in code-mixed Hinglish and demonstrates the practical effectiveness of transformer-based approaches in multilingual NLP applications.

References

Sitaram, S., Chandu, K., Rallabandi, S. K., & Black, A. W. (2019). A survey of code-switched speech and language processing. *IEEE Signal Processing Magazine, 36*(6), 92–103.

Sengupta, A., Suresh, T., Akhtar, M. S., & Chakraborty, T. (2024). A comprehensive understanding of code-mixed language semantics using hierarchical transformer. *IEEE Transactions on Computational Social Systems, 11*(3), 4139–4148.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics.

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., ... & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. *In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440–8451). Association for Computational Linguistics.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *In Advances in Neural Information Processing Systems 30* (pp. 5998–6008). Neural Information Processing Systems Foundation.

Young, T., Hazarika, D., Poria, S., & Cambria, E. (2018). Recent trends in deep learning-based natural language processing. *IEEE Computational Intelligence Magazine, 13*(3), 55–75.

Cambria, E., Das, D., Bandyopadhyay, S., & Feraco, A. (2017). Affective computing and sentiment analysis. *IEEE Intelligent Systems, 32*(2), 18–22.

Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. *In Proceedings of the 16th Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 2227–2237). Association for Computational Linguistics.

Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. *In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* (pp. 328–339). Association for Computational Linguistics.

Winata, G. I., Madotto, A., Wu, C.-S., & Fung, P. (2019). Code-switched language models using neural networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.

Parshad, R. D., Bhowmick, S., Chand, V., Kumari, N., & Sinha, N. (2016). What is India speaking? Exploring the "Hinglish" invasion. *Physica A: Statistical Mechanics and its Applications*, *449*, 375–389.

Pratapa, A., Choudhury, M., & Sitaram, S. (2018). Word embeddings for code-mixed language processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3067–3072). Association for Computational Linguistics.

Aguilar, G., & Solorio, T. (2020). From English to code-switching: Transfer learning with strong morphological clues. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8033–8044). Association for Computational Linguistics.

Bedi, M., Kumar, S., Akhtar, M. S., & Chakraborty, T. (2023). Multi-modal sarcasm detection and humor classification in code-mixed conversations. *IEEE Transactions on Affective Computing*, *14*(2), 1363–1375.

Upadhyay, S., Faruqui, M., Dyer, C., & Roth, D. (2016). Cross-lingual models of word embeddings: An empirical comparison. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1661–1670). Association for Computational Linguistics.

Nikitha, M., Savaram, P., Bandu, S., & Saketh. (2024). Semantic role labeling for code-switched data. In *2024 3rd Edition of IEEE Delhi Section Flagship Conference (DELCON)* (pp. 1–5). IEEE.

Shruthi, N., & Sooda, K. (2025). IndicBART for translating code-mixed Kannada-English sentences into Kannada: An encoder-decoder transformer approach. In *2025 5th International Conference on Intelligent Technologies (CONIT)* (pp. 1–6). IEEE.

Darak, T., Samaddar, U., Nossam, S. C., & Nair, P. C. (2025). Cross-lingual transfer learning for low-resource language sentiment analysis. In *2025 International Conference on Communication, Computer, and Information Technology (IC3IT)* (pp. 1–8).

Punetha, N., & Jain, G. (2026). Analyzing sentiment dynamics with the Hawk-Dove model for Hinglish language text via a mathematical optimization approach. *SN Computer Science*, *7*(2).

Boyina, K., Reddy, G. M., Akshita, G., & Nair, P. C. (2024). Zero shot and few-shot learning for Telugu news classification: A large language model approach. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)* (pp. 1–7). IEEE.